Chapter 4: Estimating and Measures for Performance Evaluation

Bias-Variance Tradeoff in Machine Learning

1. Bias

- Bias refers to the error introduced by approximating a real-world problem with a simplified model.
- High bias means the model **fails to capture** the underlying patterns in the data.
- Typically associated with **underfitting**.
- Example: A **linear regression** model used to fit a complex, nonlinear dataset.

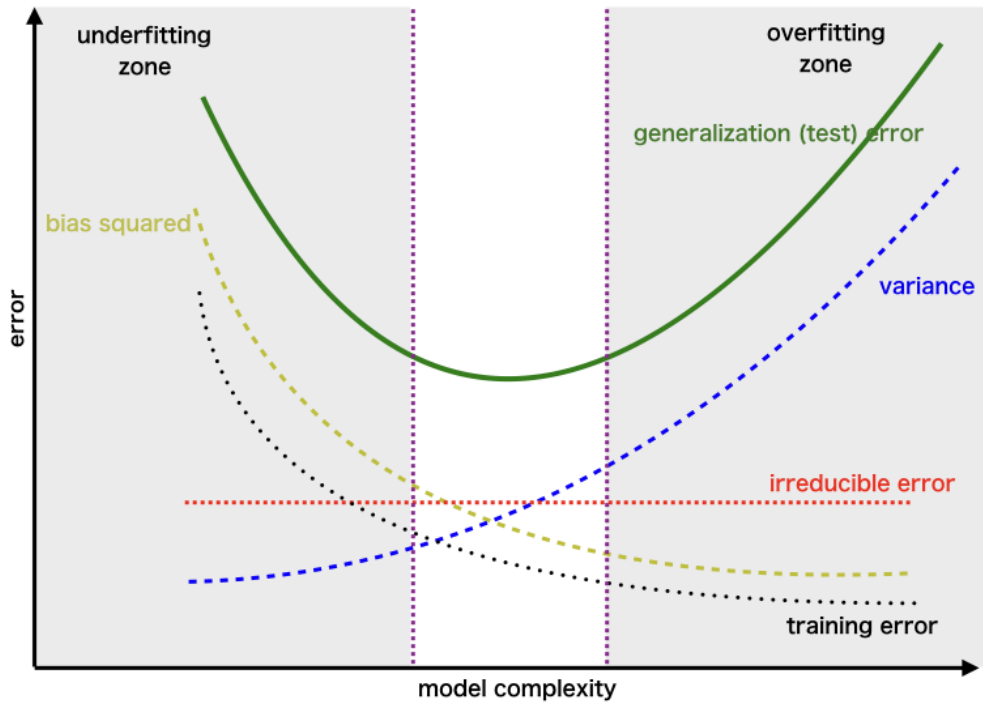
2. Variance

- Variance refers to the model's **sensitivity to small fluctuations** in the training data.
- High variance means the model **fits the training data too closely**, including noise.
- Typically associated with **overfitting**.
- Example: A **deep neural network** with too many parameters trained on a small dataset.

3. Relationship Between Bias and Variance

- Bias and variance contribute to the **total prediction error**:

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$



Bias-Variance Tradeoff Explanation (with Reference to the Image)

The **bias-variance tradeoff** is a fundamental concept in machine learning that describes the balance between a model's ability to generalize well to unseen data. This image visually represents the tradeoff by showing how different types of errors (bias, variance, training error, and test error) change with **model complexity**.

---

1. Axes Explanation

- **X-Axis (Model Complexity):** Represents how complex the model is, ranging from simple (left) to highly complex (right).
- **Y-Axis (Error):** Represents the magnitude of error in predictions. Lower error is better.

---

2. Key Components in the Graph

(A) Bias² (Yellow Dashed Line)

- Bias measures how much the model’s predictions deviate from the true values.

- **High bias** (left side) means the model is too simple to capture patterns in data, leading to **underfitting**.
- As model complexity increases, **bias decreases** because the model can capture more patterns.

(B) Variance (Blue Dashed Line)

- Variance measures how sensitive the model is to fluctuations in training data.
- **Low variance** (left side) means the model is stable but may be too simple.
- **High variance** (right side) means the model is too sensitive, causing **overfitting**.
- As complexity increases, variance **rises**, making the model more dependent on training data.

(C) Training Error (Black Dotted Line)

- Training error is the error the model makes on the training dataset.
- **Always decreases as model complexity increases** because a more complex model can fit the training data better.

(D) Generalization (Test) Error (Green Solid Line)

- Represents the model's performance on unseen data.
- Has a **U-shaped curve**:
    - **Left side (Underfitting):** High test error due to high bias.
    - **Right side (Overfitting):** High test error due to high variance.
    - **Middle (Optimal Complexity):** Lowest test error, balancing bias and variance.

(E) Irreducible Error (Red Horizontal Line)

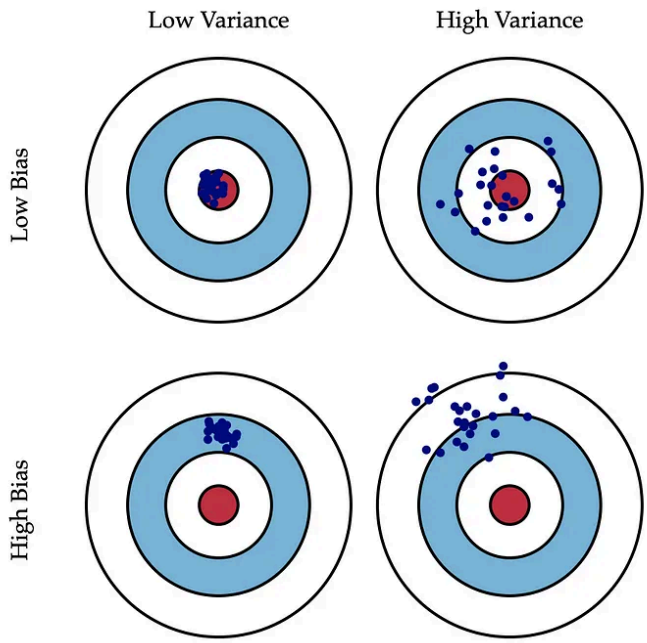
- Represents noise or randomness in the data that no model can eliminate.
- Stays constant because no model can predict perfectly due to inherent uncertainty.

---

3. Understanding the Tradeoff

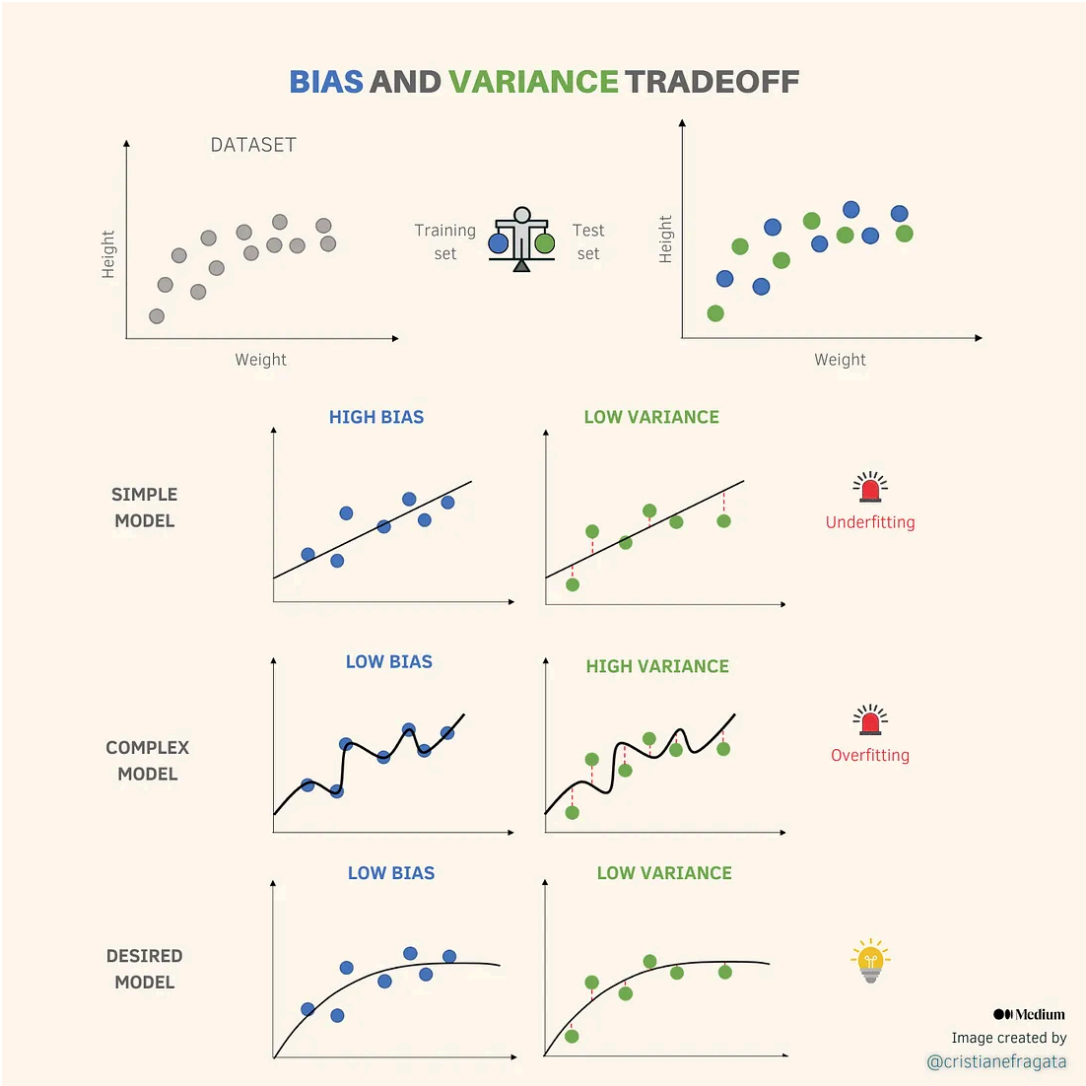
- **Underfitting (Left Side, Grey Zone)**
    - Occurs when a model is too simple to capture data patterns.
    - High bias, low variance.
    - Both training and test errors are high.
- **Overfitting (Right Side, Grey Zone)**

    - Occurs when a model is too complex and captures noise in the training data.
    - Low bias, high variance.
    - Training error is low, but test error increases.
- **Optimal Model Complexity (Middle)**
    - Balances bias and variance.
    - Achieves the **lowest test error**, ensuring good generalization.
- **Bias-Variance Tradeoff:**
    - Increasing model complexity **reduces bias** but **increases variance**.
    - Simplifying the model **reduces variance** but **increases bias**.
    - The goal is to find a balance where both are minimized for optimal performance.



4. Tradeoff Between Bias and Variance

Model Complexity	Bias	Variance	Generalization
Too Simple (Underfitting)	High	Low	Poor
Optimal (Balanced Model)	Low	Low	Good
Too Complex (Overfitting)	Low	High	Poor



5. Strategies to Address Bias-Variance Tradeoff

To achieve a good bias-variance tradeoff, consider:

- **Regularization (L1, L2, Dropout)**: Helps prevent overfitting by penalizing complexity.
- **More training data**: Reduces variance and helps the model generalize better.
- **Cross-validation**: Helps assess model performance on unseen data.
- **Choosing the right model**: A simpler model works well for small datasets, while complex models need more data.

.A combination of these techniques ensures the model generalizes well to unseen data, avoiding both underfitting and overfitting.

4.2. Cross-Validation and Resampling Methods

Cross-validation and resampling methods are essential techniques in machine learning and statistics to assess model performance, prevent overfitting, and ensure generalization to unseen data. These methods split data into training and validation sets to evaluate model accuracy.

4.2.1. The Holdout Method

The **holdout method** is the simplest form of cross-validation. The dataset is split into two or three subsets:

1. **Training set:** Used to train the model.
2. **Validation set (optional):** Used to tune hyperparameters.
3. **Test set:** Used to evaluate the model's performance.

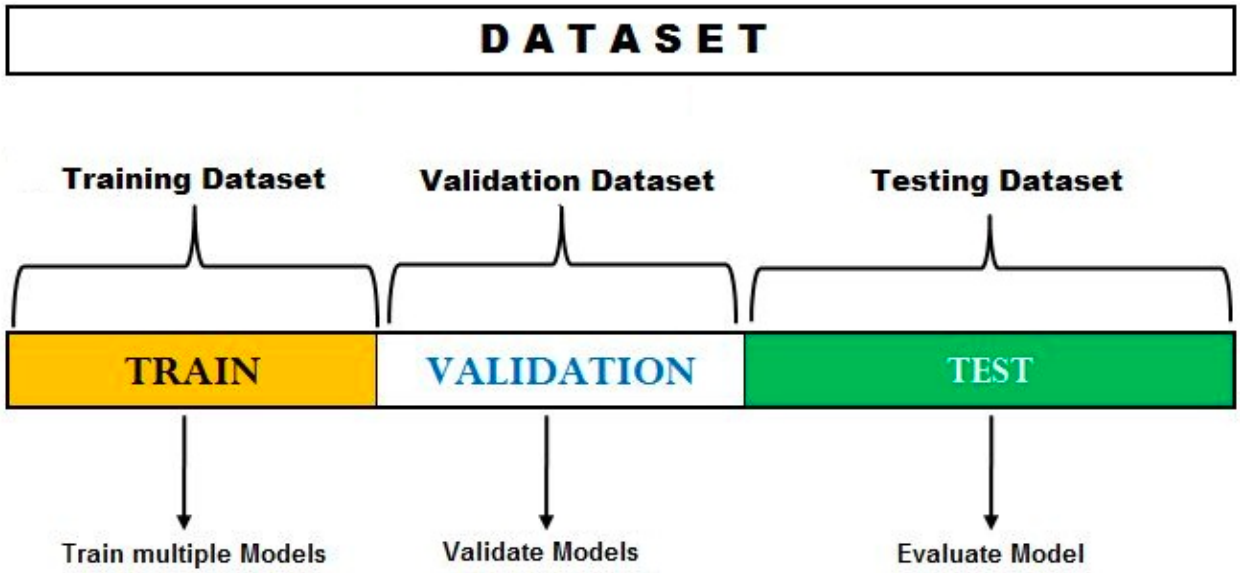
Advantages:

- Simple and fast.
- Suitable for large datasets.

Disadvantages:

- Performance may vary based on how data is split.
- Less effective for small datasets as it does not utilize all data.

Example: Splitting data into 80% training and 20% test.



4.2.2. k-Fold Cross-Validation

In **k-Fold Cross-Validation**, the dataset is divided into **k equal-sized subsets (folds)**. The model is trained **k times**, each time using **k-1 folds for training** and **1 fold for testing**.

Steps:

1. Shuffle and split the dataset into k subsets.
2. Train the model on **k-1 folds** and test it on the remaining fold.
3. Repeat the process k times, each time using a different fold as the test set.
4. Compute the average performance across all k iterations.

Advantages:

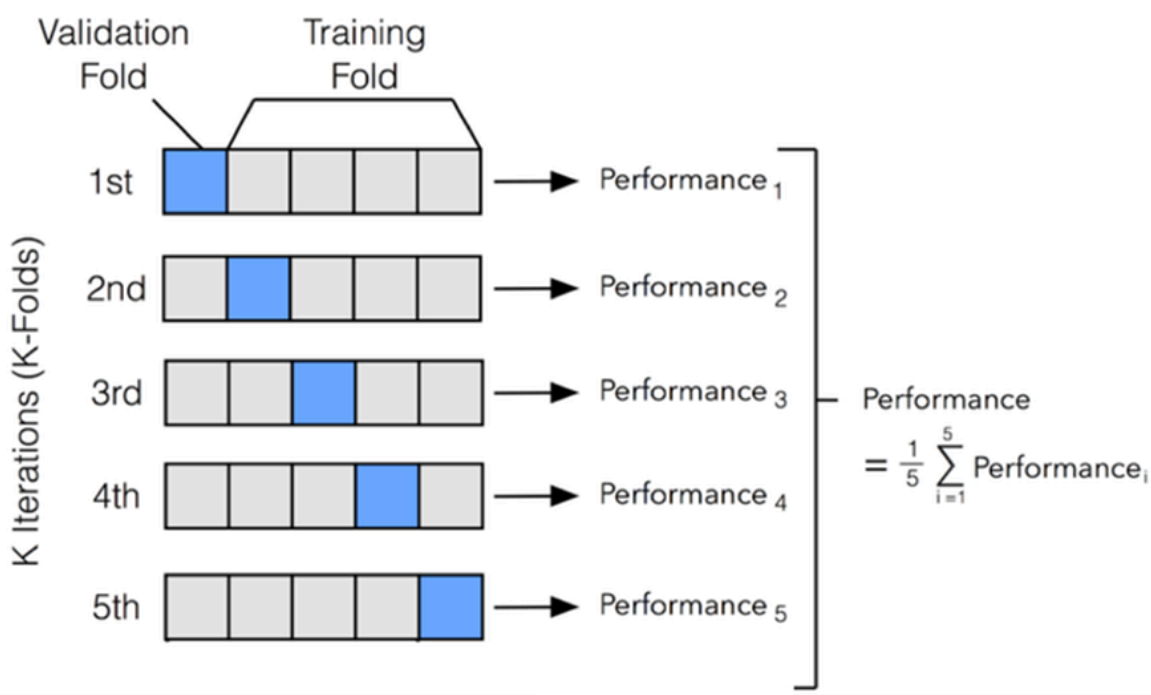
- Reduces variance by training multiple models.
- Works well for small datasets.
- More reliable estimate of model performance.

Disadvantages:

- Computationally expensive for large k-values.

Common values:

- **k = 5 or 10** is typically used in practice.



4.2.3. Bootstrap

Bootstrapping Main Ideas!!!

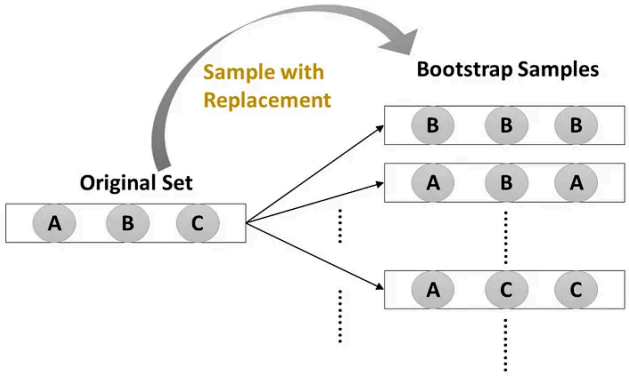
What is Bootstrap?

The **bootstrap** is a powerful statistical technique used to estimate the distribution of a statistic (e.g., mean, median, variance) by repeatedly sampling from the dataset **with replacement**. It is particularly useful when the underlying distribution of the data is unknown or when traditional analytical methods are difficult to apply.
The bootstrap method was introduced by Bradley Efron in 1979 and has since become a fundamental tool in statistics and data science.

How Does Bootstrap Work?

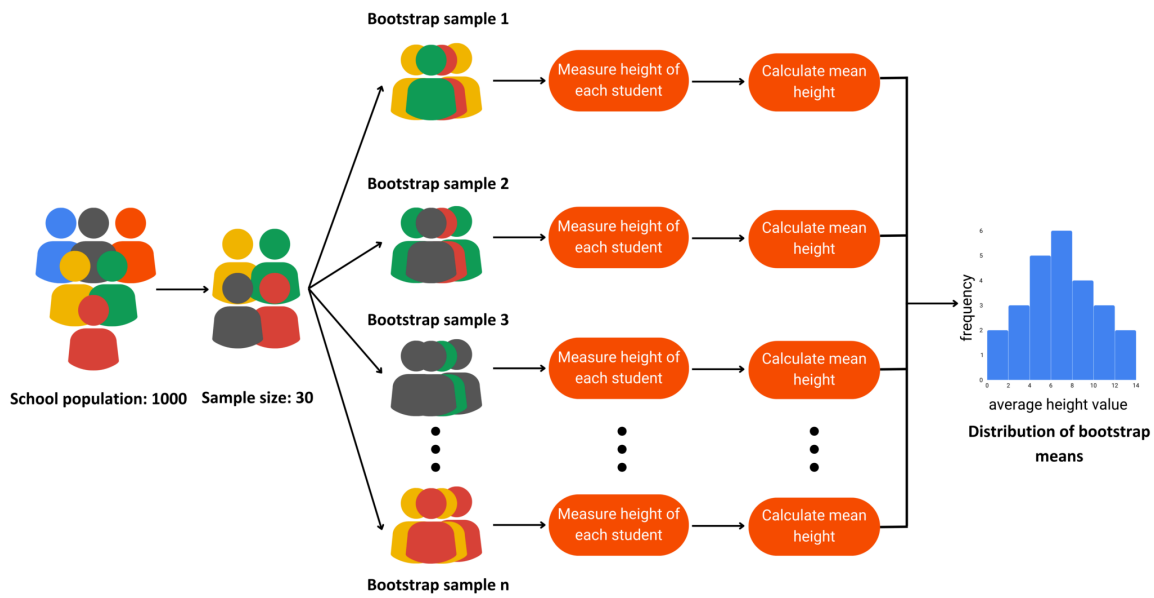
The bootstrap method involves the following steps:

1. **Resample with Replacement**:
   - From the original dataset of size n, randomly sample n observations **with replacement**. This creates a "bootstrap sample."



2. **Compute the Statistic**:
   - Calculate the statistic of interest (e.g., mean, median) for the bootstrap sample.
3. **Repeat**:
   - Repeat the resampling and computation process many times (e.g., 1,000 or 10,000 times) to create a distribution of the statistic.
4. **Analyze the Distribution**:
   - Use the distribution of the computed statistics to estimate properties like:

- **Standard Error**: The standard deviation of the bootstrap distribution.
- **Confidence Intervals**: The range within which the true statistic lies with a certain probability (e.g., 95%).



4.3 Ensemble Method:

Ensemble methods combine multiple models to improve accuracy and robustness. The key ensemble techniques include bagging and boosting, with Random Forest and AdaBoost/XGBoost as their representatives.

4..3.1 Bagging

Bagging, short for **Bootstrap Aggregating**, is an ensemble learning technique designed to improve the stability and accuracy of machine learning models. It works by training multiple models on different subsets of the training data (created by random sampling with replacement) and then combining their predictions. Bagging is particularly effective for reducing variance and preventing overfitting, especially for high-variance models like decision trees.

Key Concepts of Bagging

1. **Bootstrap Sampling**:

    - Randomly sample the training data with replacement to create multiple subsets (bootstrap samples).
    - Each subset is the same size as the original dataset but may contain duplicate samples.
2. **Model Training**:
    - Train a base model (e.g., decision tree) on each bootstrap sample.
3. **Aggregation**:
    - Combine the predictions of all models:
        - For **classification**: Use majority voting (the most frequent prediction).
        - For **regression**: Use averaging (the mean of all predictions).
4. **Out-of-Bag (OOB) Error (Optional)**:
    - For each model, evaluate its performance on the data points not included in its bootstrap sample (OOB data).
    - This provides an estimate of the model's performance without needing a separate validation set.

4.3.2 Boosting

Boosting is an ensemble learning technique designed to **improve the accuracy and reduce bias** in machine learning models. Unlike bagging, which builds independent models in parallel, **boosting trains models sequentially**, where each new model focuses on correcting the errors made by the previous models. This process enhances the predictive power of weak learners, making boosting highly effective for complex datasets.

🔹 Key Concepts of Boosting

1. Sequential Learning:

- Models are trained **one after another** instead of independently.
- Each new model **corrects the mistakes** made by the previous ones.

2. Weighted Sample Training:

- Misclassified samples receive **higher weights** so that subsequent models pay more attention to them.
- Correctly classified samples get lower weights.

3. Model Combination:

- The final prediction is a **weighted sum of all models' outputs**.
- Weights are assigned based on each model's performance.

🔹 How Boosting Works (Step by Step)

1️⃣ **Initialize Model:** Start with equal weights for all training samples.
2️⃣ **Train Weak Learner:** Fit a weak model (e.g., a shallow decision tree) to the data.
3️⃣ **Calculate Errors:** Identify misclassified samples.
4️⃣ **Adjust Weights:** Increase the weights of misclassified samples to give them more importance.
5️⃣ **Train Next Model:** Fit another weak learner on the updated dataset.
6️⃣ **Repeat Process:** Continue iterating until reaching the desired number of models.
7️⃣ **Final Prediction:** Combine all weak learners using weighted voting (classification) or averaging (regression).

🔹 Types of Boosting Algorithms

1. AdaBoost (Adaptive Boosting)

- Uses simple weak learners (often decision stumps).
- Increases the weight of misclassified samples.
- Combines weak learners using a weighted majority vote.
  AdaBoost, Clearly Explained

2. Gradient Boosting (XGBoost)

- Instead of assigning weights, **fits the next model on residual errors** (differences between actual and predicted values).
- Uses gradient descent to minimize loss.

XGBoost Part 1 (of 4): Regression

📊 Key Differences Between Bagging and Boosting

Feature	Bagging	Boosting (AdaBoost, XGBoost)
Training	Parallel (independent models)	Sequential (each model corrects the previous one)
Goal	Reduce variance	Reduce bias
Overfitting	Less prone	More prone (if not tuned)
Performance	More stable	Higher accuracy (if optimized)
Computation	Faster (parallel training)	Slower (sequential training)

📊 Key Differences: AdaBoost vs. XGBoost

Feature	AdaBoost	XGBoost
Base Model	Decision stumps (shallow trees)	Deeper decision trees
Learning Type	Adaptive boosting	Gradient boosting

Computation	Slower	Faster (parallelized)
Regularization	No regularization	L1 & L2 regularization
Handling Missing Data	Doesn't handle well	Handles internally
Noise Sensitivity	More sensitive to noise	More robust
Overfitting Risk	Higher	Lower (with tuning)
Use Case	Small to medium datasets, simple problems	Large datasets, complex patterns

4.4 Measuring Classifier Performance: Confusion Matrix

Evaluating the performance of a classification model is essential to ensure its effectiveness. The **confusion matrix** is a fundamental tool used to assess classification results by comparing actual and predicted labels. Based on the confusion matrix, we can calculate key performance metrics such as **accuracy, precision, recall, F1-score, sensitivity, and specificity**.

📌 4.4.1 Confusion Matrix

A **confusion matrix** is a table that summarizes the performance of a classification model. It consists of four key values:

Actual \ Predicted	Positive (1)	Negative (0)
Positive (1)	True Positive (**TP**)	False Negative (**FN**)

Negative (0)	False Positive (**FP**)	True Negative (**TN**)

🔹 Key Terms in a Confusion Matrix

- **True Positive (TP)**: The model correctly predicts a positive class.
- **False Negative (FN)**: The model incorrectly predicts a negative class for an actual positive instance.
- **False Positive (FP)**: The model incorrectly predicts a positive class for an actual negative instance.
- **True Negative (TN)**: The model correctly predicts a negative class.

📌 4.4.1 Classification Accuracy and Misclassification Costs

🔹 Classification Accuracy

- **Formula:**

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- Measures the proportion of correctly classified samples.
- **Limitations:** Accuracy can be misleading in imbalanced datasets (e.g., if 90% of the data belongs to one class, a model predicting only that class achieves 90% accuracy but is useless).

📌 4.4.2 Recall, Precision, and F1 Score

🔹 Recall (Sensitivity or True Positive Rate - TPR)

- Measures how many actual positive cases the model correctly identified.
- **Formula:**

$$Recall = \frac{TP}{TP + FN}$$

- **High Recall = Few False Negatives (FN)**
- **Use case:** Important when **missing positive cases is costly** (e.g., detecting cancer).

🔹 Precision (Positive Predictive Value - PPV)

- Measures how many predicted positive cases were actually positive.
- **Formula:**

$$Precision = \frac{TP}{TP + FP}$$

- **High Precision = Few False Positives (FP)**
- **Use case:** Important when **false positives are costly** (e.g., spam detection).

🔹 F1 Score (Harmonic Mean of Precision and Recall)

- A balance between precision and recall, useful for imbalanced datasets.
- **Formula:**

$$\begin{aligned} \text{F1 Score} &= \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}} \\ &= \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \end{aligned}$$

- **High F1 Score = Good balance between Precision & Recall**
- **Use case:** Used when **both FP and FN are important** (e.g., fraud detection).

📌 Sensitivity and Specificity

🔹 Sensitivity (Same as Recall - TPR)

- Measures the model's ability to correctly identify positive cases.
- **Formula:**

$$SENSITIVITY = \frac{TP}{TP + FN}$$

- High sensitivity means fewer false negatives.

🔹 Specificity (True Negative Rate - TNR)

- Measures the model's ability to correctly identify negative cases.
- **Formula:**

$$\text{Specificity} = \frac{\text{True negative}}{\text{True negative + false positive}}$$

GEEKYMEDICS.COM

- High specificity means fewer false positives.
- **Use case:** Important in medical tests where **false positives should be minimized** (e.g., ensuring a person is actually sick before prescribing expensive or harmful treatment).

📊 Summary of Performance Metrics

Metric	Formula	Interpretation	Best Use Case
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	Overall correctness	When both classes are balanced and equally important
Precision	$\frac{TP}{TP+FP}$	Correctness of positive predictions	When false positives are costly (e.g., fraud detection)
Recall (Sensitivity, TPR)	$\frac{TP}{TP+FN}$	How well actual positives are detected	When false negatives are costly (e.g., medical diagnosis)
F1 Score	$2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$	Balances precision and recall	When dataset is imbalanced
Specificity (TNR)	$\frac{TN}{TN+FP}$	How well actual negatives are detected	When false positives are costly (e.g., medical screening)

🚀 When to Use Different Metrics?

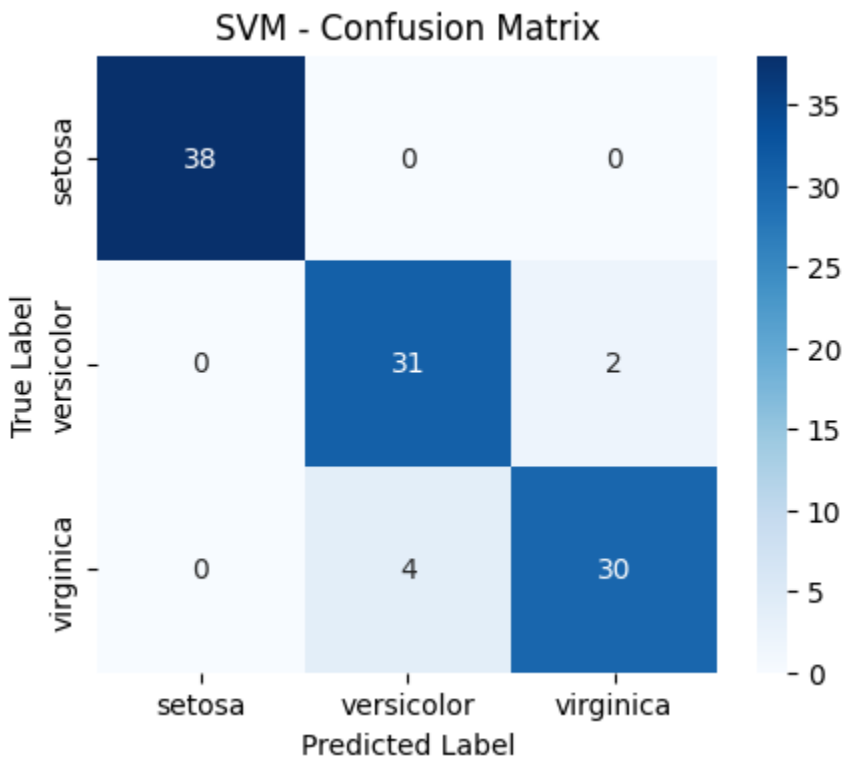
✅ **Accuracy**: Use when both classes are equally important and dataset is balanced.
✅ **Precision**: Use when false positives are costly (e.g., spam detection).
✅ **Recall (Sensitivity)**: Use when false negatives are costly (e.g., medical diagnosis).
✅ **F1 Score**: Use when the dataset is imbalanced and both false positives & false negatives matter.
✅ **Specificity**: Use when false positives must be minimized (e.g., medical screening).

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import (
```

```
    confusion_matrix, accuracy_score, precision_score, recall_score,
f1_score, classification_report
)

# Load dataset
iris = datasets.load_iris()
X = iris.data
y = iris.target

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.7)

# Train SVM model
svm_model = SVC(kernel='linear', random_state=42)
svm_model.fit(X_train, y_train)

# Make predictions
y_pred = svm_model.predict(X_test)

# Compute evaluation metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='macro')  # macro for
multi-class
recall = recall_score(y_test, y_pred, average='macro')
f1 = f1_score(y_test, y_pred, average='macro')

# Print evaluation metrics
print(f"🔹 **SVM Model Performance** 🔹")
print(f"Accuracy: {accuracy:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
print(f"F1 Score: {f1:.2f}")


# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

# Plot Confusion Matrix
plt.figure(figsize=(5, 4))
```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=iris.target_names, yticklabels=iris.target_names)
plt.title('SVM - Confusion Matrix')
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.show()
```

🔹 **SVM Model Performance** 🔹
Accuracy: 0.94
Precision: 0.94
Recall: 0.94
F1 Score: 0.94



🔹 Question 1 (Basic Confusion Matrix Calculation)

A binary classification model is tested on **100** samples. The confusion matrix is given below:

Actual \ Predicted	Positive (P1)	Negative (P2)
Positive (A1)	50	10
Negative (A2)	5	35

Calculate the following metrics:

1. **Accuracy**
2. **Precision**
3. **Recall**
4. **F1-Score**
5. **Specificity**

🔹 Question 2 (Real-world Scenario - Spam Detection)

A machine learning model is used for detecting spam emails. The confusion matrix for the classification is:

Actual \ Predicted	Spam (P1)	Not Spam (P2)
Spam (A1)	80	20
Not Spam (A2)	10	90

Calculate the following performance metrics:

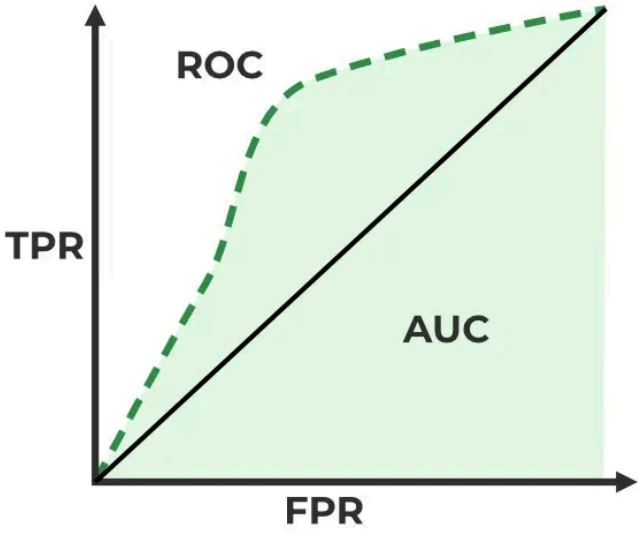
1. **Accuracy of the model.**
2. **Precision for spam detection.**
3. **Recall (Sensitivity) for spam emails.**
4. **F1-Score.**
5. **Specificity (True Negative Rate).**

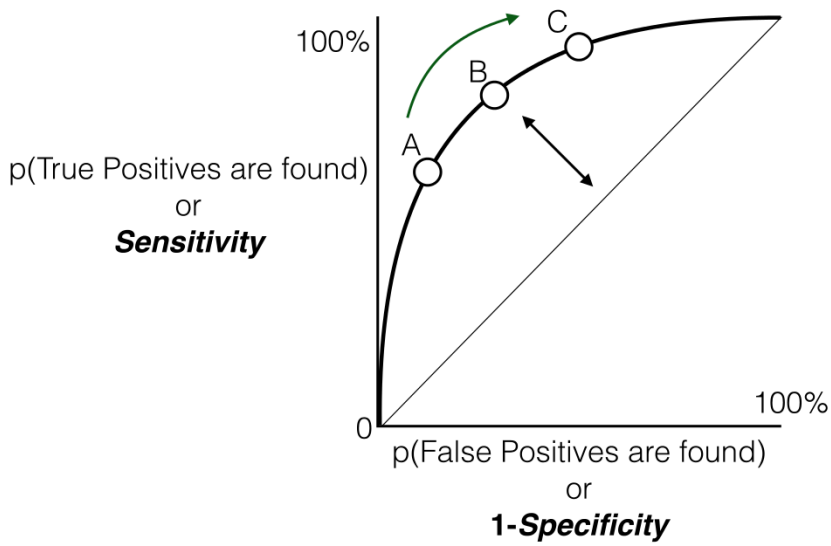
AUC-ROC Curve in Machine Learning

The **Receiver Operating Characteristic (ROC) curve** and the **Area Under the Curve (AUC)** are widely used metrics to compare the performance of classification models, particularly in **imbalanced datasets** or cases where predicting probabilities is important.

1. What is the AUC-ROC Curve?

- **ROC Curve (Receiver Operating Characteristic Curve)**:
  - A graph that plots **True Positive Rate (TPR)** against **False Positive Rate (FPR)** at various classification thresholds.
  - It shows how well the model distinguishes between two classes.
- **AUC (Area Under the Curve)**:
  - Measures the area under the ROC curve.
  - Higher AUC means better model performance.
  - **AUC = 1** → Perfect model
  - **AUC = 0.5** → Model is guessing randomly
  - **AUC < 0.5** → Model is worse than random guessing





Let's work through a simple example to calculate the AUC using the trapezoidal rule.

Example Data Points

Suppose we have the following points on the ROC curve:

Threshold	False Positive Rate (FPR)	True Positive Rate (TPR)
1	0.0	0.0
2	0.1	0.4
3	0.3	0.7
4	0.6	0.8
5	1.0	1.0

These points form the ROC curve.

Step-by-Step AUC Calculation Using the Trapezoidal Rule

The formula for the area of a trapezoid between two points is:

$$\text{Area} = (\text{FPR}_{i+1} - \text{FPR}_i) \times \frac{(\text{TPR}_i + \text{TPR}_{i+1})}{2}$$

Segment 1: Between FPR = 0.0 and FPR = 0.1

- Width: $0.1 - 0.0 = 0.1$
- Average Height: $\frac{0.0+0.4}{2} = 0.2$
- Area: $0.1 \times 0.2 = 0.02$

Segment 2: Between FPR = 0.1 and FPR = 0.3

- Width: $0.3 - 0.1 = 0.2$
- Average Height: $\frac{0.4+0.7}{2} = 0.55$
- Area: $0.2 \times 0.55 = 0.11$

Segment 3: Between FPR = 0.3 and FPR = 0.6

- Width: $0.6 - 0.3 = 0.3$
- Average Height: $\frac{0.7+0.8}{2} = 0.75$
- Area: $0.3 \times 0.75 = 0.225$

Segment 4: Between FPR = 0.6 and FPR = 1.0

- Width: $1.0 - 0.6 = 0.4$
- Average Height: $\frac{0.8+1.0}{2} = 0.9$
- Area: $0.4 \times 0.9 = 0.36$

Total AUC Calculation

Sum the areas of all segments:

$$\text{AUC} = 0.02 + 0.11 + 0.225 + 0.36 = 0.715$$

Interpretation

An AUC of **0.715** means that, on average, the model has a 71.5% chance of ranking a randomly chosen positive instance higher than a randomly chosen negative instance.

This step-by-step process illustrates how the AUC is calculated from the ROC curve using the trapezoidal rule.

Example:

To compare the performance of SVM (Support Vector Machine) and Decision Tree classifiers using ROC AUC curves on the Iris dataset, here's the general procedure in Python using `sklearn`:

1. Load the Iris Dataset:

The Iris dataset is available directly from `sklearn`'s datasets module.

2. Train SVM and Decision Tree Models:

You'll train both SVM and Decision Tree classifiers using the dataset.

3. Evaluate Models Using ROC AUC:

After training the models, you'll use the ROC curve and AUC to evaluate their performance.

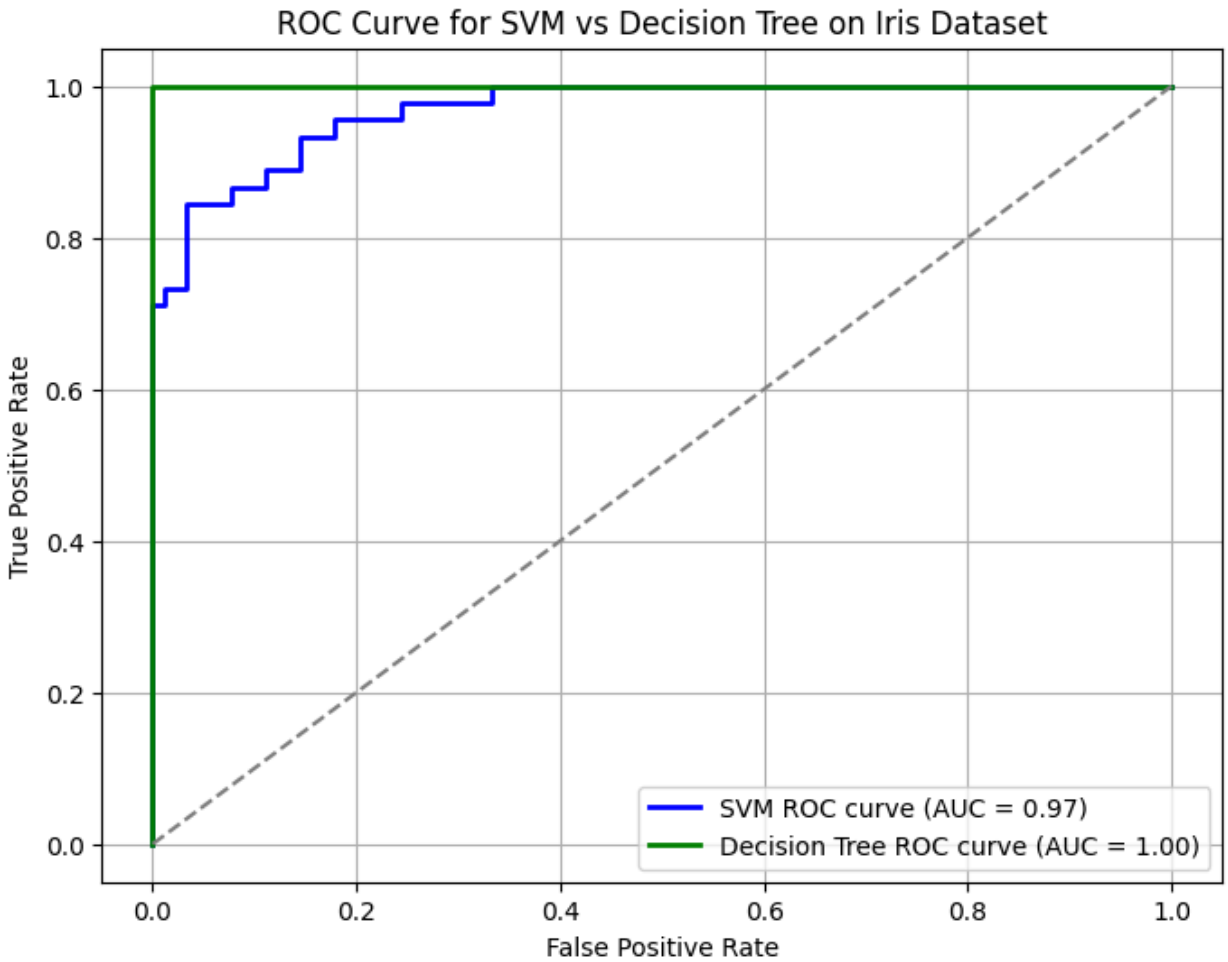
Here is a Python code snippet to achieve this:

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn import datasets

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import label_binarize

from sklearn.svm import SVC

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import roc_curve, auc

from sklearn.multiclass import OneVsRestClassifier



# Load Iris dataset

iris = datasets.load_iris()

X = iris.data
```

```
y = iris.target


# Binarize the target labels for multiclass ROC AUC

y_bin = label_binarize(y, classes=[0, 1, 2])


# Split data into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(X, y_bin,
test_size=0.3, random_state=42)


# Train SVM and Decision Tree classifiers (OneVsRest for multiclass)

svm = OneVsRestClassifier(SVC(kernel='linear', probability=True,
random_state=42))

dtree = OneVsRestClassifier(DecisionTreeClassifier(random_state=42))


# Fit the models

svm.fit(X_train, y_train)

dtree.fit(X_train, y_train)


# Get predicted probabilities for the test set

y_pred_svm = svm.predict_proba(X_test)

y_pred_dtree = dtree.predict_proba(X_test)


# Compute ROC curve and AUC for both models

fpr_svm, tpr_svm, _ = roc_curve(y_test.ravel(), y_pred_svm.ravel())
```

```
roc_auc_svm = auc(fpr_svm, tpr_svm)


fpr_dtree, tpr_dtree, _ = roc_curve(y_test.ravel(), y_pred_dtree.ravel())

roc_auc_dtree = auc(fpr_dtree, tpr_dtree)


# Plot the ROC curve for both models

plt.figure(figsize=(8, 6))


plt.plot(fpr_svm, tpr_svm, color='blue', lw=2, label=f'SVM ROC curve (AUC
= {roc_auc_svm:.2f})')

plt.plot(fpr_dtree, tpr_dtree, color='green', lw=2, label=f'Decision Tree
ROC curve (AUC = {roc_auc_dtree:.2f})')


# Random classifier line

plt.plot([0, 1], [0, 1], color='gray', linestyle='--')


# Formatting

plt.title('ROC Curve for SVM vs Decision Tree on Iris Dataset')

plt.xlabel('False Positive Rate')

plt.ylabel('True Positive Rate')

plt.legend(loc='lower right')

plt.grid(True)


# Show the plot
```

```
plt.show()



# Print the AUC values

print(f"AUC for SVM: {roc_auc_svm:.2f}")

print(f"AUC for Decision Tree: {roc_auc_dtree:.2f}")
```



4. How to Interpret the Comparison?

1. **Compare AUC values**:
    - The **higher** the AUC, the better the model distinguishes between classes.

    - Example:
        - **SVM AUC = 0.97**
        - **Decision Tree AUC = 1.00**
        - **Conclusion**: Decision Tree is performing better than SVM.
2. **Compare ROC Curves**:
    - The model with the **higher curve** (closer to the top-left) is better.
    - A model that is close to the diagonal line (`AUC ≈ 0.5`) is no better than random guessing.